

第二章 基础介绍

大语言模型是指在海量无标注文本数据上进行预训练得到的大型预训练语言模型，例如 GPT-3 [23]，PaLM [33] 和 LLaMA [34]。目前大语言模型所需要具有的最小参数规模还没有一个明确的参考标准，但是大语言模型通常是指参数规模达到百亿、千亿甚至万亿的模型；也有部分工作认为经过大规模数据预训练（显著多于传统预训练模型如 BERT 所需要的训练数据）的数十亿参数级别的模型也可以称之为大语言模型（如 LLaMA-2 7B）。对于大语言模型，本书泛指具有超大规模参数或者经过超大规模数据训练所得到的语言模型。与传统语言模型相比，大语言模型的构建过程涉及到更为复杂的训练方法，进而展现出了强大的自然语言理解能力和复杂任务求解能力（通过文本生成的形式）。为了帮助读者了解大语言模型的工作原理，本部分将介绍大语言模型的构建过程、扩展法则（Scaling Law）、涌现能力（Emergent Abilities），然后将介绍 GPT 系列模型的研发历程。

2.1 大语言模型的构建过程

本部分内容将概要介绍大语言模型的构建过程，为刚进入该领域的读者对于大语言模型的研发建立一个初步的认识。从机器学习的观点来说，神经网络是一种具有特定模型结构的函数形式，而大语言模型则是一种基于 Transformer 结构的神经网络模型。因此，可以将大语言模型看作一种拥有大规模参数的函数，它的构建过程就是使用训练数据对于模型参数的拟合过程。尽管所采用的训练方法与传统的机器学习模型（如多元线性回归模型的训练）可能存在不同，但是本质上都是在做模型参数的优化。大语言模型的优化目标更加泛化，不仅仅是为了解决某一种或者某一类特定任务，而是希望能够作为通用任务的求解器（如图 1.2）。为了实现这一宏大的目标，大语言模型的构建过程需要更为复杂、精细的训练方法。一般来说，这个训练过程可以分为大规模预训练和指令微调与人类对齐两个阶段，下面将进行具体介绍。

2.1.1 大规模预训练

一般来说，预训练是指使用与下游任务无关的大规模数据进行模型参数的初始训练，可以认为是为模型参数找到一个较好的“初值点”。这一思想最早在计算机视觉领域被广泛使用，通过使用大规模的图像标注数据集 ImageNet 用于初始化视觉模型的参数。在自然语言处理领域，word2vec [8] 采用了类似的预训练思想，使用无标注的文本语料训练可通用的词嵌入模型；后来被 ELMo [11]、BERT [13] 和 GPT-1 [14] 推广到训练可迁移的自然语言任务架构，逐步成为了研发大语言模型的核心技术路径。早期的预训练技术还是聚焦于解决下游某一类的特定任务，如传统的自然语言处理任务。OpenAI 在 GPT-2 [17] 的论文中，提出通过大规模文本数据的预训练实现通用任务的求解器（尽管 GPT-2 论文中所验证的实验还是主要以自然语言处理任务为主），并且将这一思路在 GPT-3 中推广到了当时最大的千亿规模。OpenAI 前首席科学家 Ilya Sutskever 在公开采访中指出大规模预训练本质上是在做一个世界知识的压缩，从而能够学习到一个编码世界知识的参数模型，这个模型能够通过解压缩所需要的知识来解决真实世界的任务。在 BERT 等传统预训练模型中，所采用的模型架构以及训练任务还比较多样。由于 GPT 系列模型的爆火，“解码器架构 + 预测下一个词”的有效性得到了充分验证，已经成为现有大语言模型主要采纳的技术路径。

为了预训练大语言模型，需要准备大规模的文本数据，并且进行严格的清洗，去除掉可能包含有毒有害的内容，最后将清洗后的数据进行词元化（Tokenization）流，并且切分成批次（Batch），用于大语言模型的预训练。由于大语言模型的能力基础主要来源于预训练数据，因此数据的收集与清洗对于模型性能具有重要的影响。收集高质量、多源化的数据以及对于数据进行严格的清洗是构建大语言模型关键能力的重中之重，需要大模型研发人员的高度关注。目前的开源模型普遍采用 2~3T 规模的词元进行预训练，并有趋势进一步扩大这一规模。这一过程对于算力需求量极高，一般来说训练百亿模型至少需要百卡规模的算力集群（如 A100 80G）联合训练数月时间（与具体的算力资源相关）；而训练千亿模型则需要千卡甚至万卡规模的算力集群，对于算力资源的消耗非常惊人。

尽管整体的预训练技术框架非常直观，但是实施过程中涉及到大量需要深入探索的经验性技术，如数据如何进行配比、如何进行学习率的调整、如何早期发现模型的异常行为等。预训练过程需要考虑各种实施细节，而这些细节有很多并没有公开发表的经验可循，需要研发人员具有丰富的训练经验和异常处理能力，避

免大规模训练开始以后进行回退和反复迭代，从而减少算力资源的浪费，提升训练成功的几率。大语言模型的研发看似是一个算力需求型的工程，实际上相关人才是最重要的。可以说，一个大语言模型项目的核心训练人员的能力最后会决定模型的整体水平。

2.1.2 指令微调与人类对齐

经过大规模数据预训练后的语言模型已经具备较强的模型能力，能够编码丰富的世界知识，但是由于预训练任务形式所限，这些模型更擅长于文本补全，并不适合直接解决具体的任务。尽管可以通过上下文学习（In-Context Learning, ICL）等提示学习技术进行适配，但是模型自身对于任务的感知与解决能力仍然较为局限。这里做一个简单的类比。预训练后的模型就像进入工作岗位的毕业生，尽管学习了很多通用的文化课，具备了一定的实习经验，但是仍然需要加强面向特定岗位的工作能力，并且深入了解工作岗位所涉及的相关要求。因此，用人单位往往需要设置特定的培训环节，对于新入职的人员针对业务场景以及所需要的技术进行专门提升。相似地，当预训练结束后，通常需要对于大语言模型进行微调与对齐，使之更好地被用于任务求解，为人类服务。

目前来说，比较广泛使用的微调技术是“指令微调”（也叫做有监督微调，Supervised Fine-tuning, SFT），通过使用任务输入与输出的配对数据进行模型训练，可以使得语言模型较好地掌握通过问答形式进行任务求解的能力。这种模仿示例数据进行学习的过程本质属于机器学习中的模仿学习（Imitation Learning）。给定一个特定任务，虽然可能存在很多解答方式，模仿学习旨在加强对于标准答案（即师傅的示范动作）的复刻学习。一般来说，指令微调很难教会大语言模型预训练阶段没有学习到的知识与能力，它主要起到了对于模型能力的激发作用，而不是知识注入作用。与预训练相比，指令微调通常来说需要的指令实例数据规模要小的多。通常来说，数十万到百万规模的指令微调数据能够有效地激发语言模型的通用任务解决能力，甚至有些工作认为数千条或者数万条高质量指令数据也能达到不错的微调效果。因此，指令微调对于算力资源的需求相对较小。一般情况下，若干台单机八卡（A100-80G）的服务器就能在一天或数天的时间内完成百亿模型的指令微调，当指令数据规模较大的时候可以进一步增加所需要的算力资源。这个过程还可以进一步加入多轮次的对话数据来增强模型的人机对话能力。

除了提升任务的解决能力外，还需要将大语言模型与人类的期望、需求以及

价值观对齐 (Alignment)，这对于大模型的部署与应用具有重要的意义。OpenAI 在 2022 年初发布了 InstructGPT [28] 的学术论文，系统地介绍了如何将语言模型进行人类对齐。具体来说，主要引入了基于人类反馈的强化学习对齐方法 RLHF (Reinforcement Learning from Human Feedback)，在指令微调后使用强化学习加强模型的对齐能力。在 RLHF 算法中，需要训练一个符合人类价值观的奖励模型 (Reward Model)。为此，需要标注人员针对大语言模型所生成的多条输出进行偏好排序，并使用偏好数据训练奖励模型，用于判断模型的输出质量。由于强化学习需要维护更多的辅助模型进行训练，通常来说对于资源的消耗会多于指令微调，但是也远小于预训练阶段所需要的算力资源。目前还有很多工作试图通过消除奖励模型的使用，或其他使用 SFT 方式来达到与 RLHF 相似的效果，从而简化模型的对齐过程。

经历上述两个过程后，大语言模型就能够具备较好的人机交互能力，通过问答形式解决人类所提出的问题。这个构建过程需要大量的算力资源支持，也需要具有良好洞察力和训练经验的研发人员进行相关技术路线的设计与执行。因此，实现具有 ChatGPT 或者 GPT-4 能力的大语言模型绝非易事，需要进行深入的探索与实践。

2.2 扩展法则

大语言模型获得成功的关键在于对“规模扩展”(Scaling)的充分探索与利用。在实现上，大语言模型采用了与小型预训练语言模型相似的神经网络结构(基于注意力机制的 Transformer 架构)和预训练方法(如语言建模)。但是通过扩展参数规模、数据规模和计算算力，大语言模型的能力显著超越了小型语言模型的能力。有趣的是，这种通过扩展所带来的性能提升通常显著高于通过改进架构、算法等方面所带来的改进。因此，建立定量的建模方法，即扩展法则 (Scaling Law)，来研究规模扩展所带来的模型性能提升具有重要的实践指导意义。在本部分，将首先介绍两种常见的语言模型扩展法则的定义，并且进一步对于扩展法则进行深入讨论。

2.2.1 KM 扩展法则

2020 年，Kaplan 等人 [15] (OpenAI 团队) 首次建立了神经语言模型性能与三个主要因素——模型规模 (N)、数据规模 (D) 和计算算力 (C) 之间的幂律关系

(Power-Law Relationship)。由于原始论文中没有给出具体的扩展法则命名，本部分内容中使用两位共同第一作者姓氏的首字母来进行命名。在给定算力预算 c 的条件下，可以近似得到以下三个基本指数公式来描述扩展法则：

$$\begin{aligned} L(N) &= \left(\frac{N_c}{N} \right)^{\alpha_N}, \quad \alpha_N \sim 0.076, N_c \sim 8.8 \times 10^{13} \\ L(D) &= \left(\frac{D_c}{D} \right)^{\alpha_D}, \quad \alpha_D \sim 0.095, D_c \sim 5.4 \times 10^{13} \\ L(C) &= \left(\frac{C_c}{C} \right)^{\alpha_C}, \quad \alpha_C \sim 0.050, C_c \sim 3.1 \times 10^8 \end{aligned} \quad (2.1)$$

这里， $L(\cdot)$ 表示用以 nat^1 为单位的交叉熵损失。其中， N_c 、 D_c 和 C_c 分别表示非嵌入参数数量、训练数据数量和实际的算力开销。为了便于讨论，我们在不影响表达和理解的情况下对于原始的公式符号进行了适度简化。这三个公式是通过模型在不同数据规模（22M 到 23B 词元）、模型规模（768M 到 1.5B 非嵌入参数）和算力规模下的性能表现拟合推导得到的。为了推导这些公式，需要约定一些基本假设：一个因素的分析不会受到其他两个因素的限制，如当变动模型参数规模的时候，需要保证数据资源是充足的。

由公式 2.1 可见，模型性能与这三个因素之间存在着较强的依赖关系，可以近似刻画为指数关系。上述公式为规模扩展效应提供了一种定量的普适建模方法。通过普适规则能够更好地探究问题的本质，排除其他复杂因素的影响与干扰（如 OpenAI 研究团队发现模型形状对于上述公式的影响并不大）。

为了便于理解扩展法则对于模型性能的影响，OpenAI 的研究团队又将这里的损失函数进一步分解为两部分 [21]，包括不可约损失（真实数据分布的熵）和可约损失（真实分布和模型分布之间 KL 散度的估计）：

$$L(x) = \underbrace{L_\infty}_{\text{不可约损失}} + \underbrace{\left(\frac{x_0}{x} \right)^{\alpha_x}}_{\text{可约损失}}, \quad (2.2)$$

这里 x 是一个占位符号，可以指代公式 2.1 中的 N 、 D 和 C 。其中，不可约损失由数据自身特征确定，无法通过扩展法则或者优化算法进行约减；模型性能的优化只能减小可约损失部分。

¹nat 用来表示以 e 为底信息量的自然对数。

2.2.2 Chinchilla 扩展法则

Hoffmann 等人 [22] (DeepMind 团队) 于 2022 年提出了一种可选的扩展法则, 旨在指导大语言模型充分利用给定的算力资源进行优化训练。通过针对更大范围的模型规模 (70M 到 16B 参数) 和数据规模 (5B 到 500B 词元) 进行实验, 研究人员拟合得到了另一种关于模型性能的幂律关系:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (2.3)$$

其中 $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$ 和 $\beta = 0.28$ 。进一步, 利用约束条件 $C \approx 6ND$ 对于损失函数 $L(N, D)$ 进行推导, 能够获得算力资源固定情况下模型规模与数据规模的最优分配方案 (如下所示):

$$N_{\text{opt}}(C) = G \left(\frac{C}{6} \right)^a, \quad D_{\text{opt}}(C) = G^{-1} \left(\frac{C}{6} \right)^b, \quad (2.4)$$

这里, $a = \frac{\alpha}{\alpha+\beta}$, $b = \frac{\beta}{\alpha+\beta}$, G 是由 A 、 B 、 α 和 β 计算得出的扩展系数。

进一步, 研究人员 [22] 发现 KM 扩展法则和 Chinchilla 扩展法则都可以近似表示成上述算力为核心的公式 (公式 2.4):

$$N_{\text{opt}} \approx C^a, D_{\text{opt}} \approx C^b, \quad (2.5)$$

即当**算力 C 给定**的情况下, 最优的模型参数规模和数据规模由指数系数 a 和 b 分别确定。可以看到, a 和 b 决定了参数规模和数据规模的资源分配优先级: 当 $a > b$ 时, 应该用更多的算力去提高参数规模; 当 $b > a$ 时, 应该用更多的算力去提高数据规模。尽管 KM 扩展法则和 Chinchilla 扩展法则具有相似的公式形式, 但是在模型规模和数据规模的扩展上存在一定的差异。随着算力预算的增加, KM 扩展法则 ($a \approx 0.73$, $b \approx 0.27$ [22]) 倾向于将更大的预算分配给模型规模的增加, 而不是分配给数据规模的增加; 而 Chinchilla 扩展法则主张两种规模参数应该以等比例关系增加 ($a \approx 0.46$, $b \approx 0.54$ [22])。

Chinchilla 扩展法则这项研究的意义并不在于给出了资源在数据规模与模型规模上的具体分配方案, 而是首次形式化指出了之前的预训练工作可能忽视了训练数据的规模扩展。例如, 具有 175B 参数的 GPT-3 仅仅使用了 300B 的词元进行训练, 所使用的数据量远远没有达到模型能够编码的最大数据容量。根据 Chinchilla 扩展法则的指导, DeepMind 的研究团队进一步训练得到了具有 70B 参数的 Chinchilla 模型, 使用大概 1.4T 的词元进行训练。虽然后续有些人借鉴 Chinchilla 模型的线

性分配比例（数据规模大概是模型参数规模的五倍），但是目前这一分配系数已经基本没有参考意义了。越来越多的工作表明，现有的预训练语言模型对于数据的需求量远高于这些扩展法则中所给出的估计规模。例如，LLaMA-2 (7B) 的模型就使用了 2T 的词元进行训练，很多更小的模型也能够通过使用超大规模的预训练数据获得较大的模型性能提升。这种现象的一个重要原因是由于 Transformer 架构具有较好的数据扩展性，到目前为止，还没有实验能够有效验证特定参数规模语言模型的饱和数据规模（即随着数据规模的扩展，模型性能不再提升）。

2.2.3 关于扩展法则的讨论

在介绍完上述两个扩展法则后，我们围绕可预测的扩展以及任务层面的可预测性展开深入讨论，以加强读者对于扩展法则的理解。

- **可预测的扩展 (Predictable Scaling)**：在实践中，扩展法则可以用于指导大语言模型的训练，通过较小算力资源可靠地估计较大算力资源投入后的模型性能，这被称为可预测的扩展 [35]。这种可预测性主要体现在两个方面：使用小模型的性能去预估大模型的性能，或者使用大模型的早期训练性能去估计训练完成后的性能。可预测扩展对于大模型训练具有两个主要的指导作用。首先，对于大语言模型来说，详细进行各种训练技巧或变体的测试需要耗费巨大的算力资源。因此，一个较为理想的经验性方法是，基于小模型获得训练经验然后应用于大模型的训练，从而减少实验成本。例如，可以训练小型代理模型来确定适合大型模型的预训练数据混合的最佳比例 [36]。其次，大语言模型的训练过程较长，经常面临着训练损失波动情况，扩展法则可以用于监控大语言模型的训练状态，如在早期识别异常性能。尽管扩展法则刻画了模型性能增长（或模型损失减少）的平滑趋势，但是指数形式的变化趋势意味着可能会出现随规模扩展的收益递减情况，即后期的扩展增益开始变得缓慢甚至停滞。根据 OpenAI 团队的一项研究表明 [21]，即使接近递减收益点（即接近不可规约的模型损失，见公式 2.2），模型表征的质量仍然能够随着规模扩展而有效提升 [21]。这一发现表明，训练大型模型对于改善下游任务的性能是非常重要的。随着模型规模的不断增加，一个潜在问题是可供用来训练大语言模型的数据量实际上是有限的，公共文本数据将很快变得“枯竭”。因此，如何在数据受限的情况下建模扩展法则，仍然具有重要的实践意义。在这一情况下，数据重复或数据合成可能有助于缓解数据稀缺问题。

- **任务层面的可预测性**。现有关于扩展法则的研究大多数是基于语言建模损失

开展的，例如预测下一个词元的平均交叉熵损失 [15]，这一度量本身是平滑的，是对于模型整体能力的宏观度量。在实践中，我们则更关注大语言模型在真实任务中的性能提升。为了建立扩展法则与模型任务性能的关联，一个基础问题就是语言建模损失的减少是否真正意味着（或对应着）真实任务上模型性能的提高 [21]。整体上来说，语言建模损失较小的模型往往在下游任务中表现更好，因为语言建模的能力可以被认为是一种模型整体能力的综合度量。然而，语言建模损失的减少并不总是意味着模型在下游任务上的性能改善。对于某些特殊任务，甚至会出现“逆向扩展”（Inverse Scaling）现象，即随着语言建模损失的降低，任务性能却出人意料地变差 [37]。总体而言，探索和描述任务层面的扩展法则更加困难，因为它可能还依赖于任务相关的信息（如任务指标、任务难度等）。根据 GPT-4 的报告 [35]，通过扩展法则可以准确预测某些任务能力（例如编码能力），但是对于有些任务的性能预测是非常困难的。此外，有些重要能力（例如上下文学习能力 [23]）根据扩展法则是不可预测的，只有当模型大小超过一定规模时才会出现，如下文所讨论的涌现能力。

2.3 涌现能力

在现有文献中 [24]，大语言模型的涌现能力被非形式化定义为“在小型模型中不存在但在大模型中出现的能力”，具体是指当模型扩展到一定规模时，模型的特定任务性能突然出现显著跃升的趋势，远超过随机水平。类比而言，这种性能涌现模式与物理学中的相变现象有一定程度的相似，但是仍然缺乏相应的理论解释以及理论证实，甚至有些研究工作对于涌现能力是否存在提出质疑 [38]。整体来说，涌现能力的提出有助于使得公众认识到大语言模型所具有的能力优势，能够帮助区分大语言模型与传统预训练语言模型之间的差异。在本书中，涌现能力用来指代大语言模型所具有的典型能力，并不关注该能力是否存在于小模型中。下面，首先在第 2.3.1 节中介绍三种具有代表性的涌现能力，随后在第 2.3.2 节中讨论涌现能力的不同观点以及可能存在争议的原因。

2.3.1 代表性的涌现能力

尽管涌现能力可以定义为解决某些复杂任务的能力水平，但我们更关注可以用来解决各种任务的普适能力。下面简要介绍大语言模型的三种典型涌现能力。

- 上下文学习 (In-context Learning, ICL) . 上下文学习能力在 GPT-3 的论文中 [23] 被正式提出。具体方式为, 在提示中为语言模型提供自然语言指令和多个任务示例 (Demonstration), 无需显式的训练或梯度更新, 仅输入文本的单词序列就能为测试样本生成预期的输出。在 GPT 系列模型中, 175B 参数的 GPT-3 模型展现出强大的上下文学习能力, 而 GPT-1 和 GPT-2 模型则不具备这种能力。此外, 上下文学习能力还取决于具体的下游任务。例如, 13B 参数的 GPT-3 模型可以在算术任务 (例如 3 位数的加减法) 上展现出上下文学习能力, 但 175B 参数的 GPT-3 模型在波斯语问答任务上甚至不能表现出良好的性能 [24]。

- 指令遵循 (Instruction Following) . 指令遵循能力是指大语言模型能够按照自然语言指令来执行对应的任务 [28, 39, 40]。为了获得这一能力, 通常需要使用自然语言描述的多任务示例数据集进行微调, 称为指令微调 (Instruction Tuning) 或监督微调 (Supervised Fine-tuning)。通过指令微调, 大语言模型可以在没有使用显式示例的情况下按照任务指令完成新任务, 有效提升了模型的泛化能力。相比于上下文学习能力, 指令遵循能力整体上更容易获得, 但是最终的任务执行效果还取决于模型性能和任务难度决定。例如, FLAN-PaLM 模型 [41] 测试了 8B、62B 以及 540B 三个参数规模的模型在指令微调之后的效果, 当参数规模达到 62B 及以上的情况, 才能够在包含 23 个复杂推理任务的 BBH 评估基准上, 展现出较好的零样本推理能力。对于规模相对较小的语言模型 (如 2B), 也可以通过使用高质量指令数据微调的方式习得一定的通用指令遵循能力 (主要是简单任务, 如文本摘要等) [42]。

- 逐步推理 (Step-by-step Reasoning) . 对于小型语言模型而言, 通常很难解决涉及多个推理步骤的复杂任务 (如数学应用题), 而大语言模型则可以利用思维链 (Chain-of-Thought, CoT) 提示策略 [25] 来加强推理性能。具体来说, 大语言模型可以在提示中引入任务相关的中间推理步骤来加强复杂任务的求解, 从而获得更为可靠的答案。在思维链的原始论文中发现 [25], 对于 62B 和 540B 参数的 PaLM 模型, 思维链提示可以提高其在算术推理基准上的效果, 但是 8B 参数的模型则很难获得提升。进一步, 思维链所带来的提升在 540B 参数的 PaLM 模型上会更加明细。此外, 思维链提示对不同任务的性能提升也不完全相同, 例如 PaLM 模型在三个数据集上产生了不同的提升幅度 (GSM8K > MAWPS > SWAMP) [25]。思维链提示特别适合帮助大语言模型解决复杂数学问题, 而具有思维链能力也是大语言模型能力的重要体现。

通常来说，很难统一界定大语言模型出现这些上述能力的临界规模（即具备某种能力的最小规模），因为能力涌现会受到多种因素或者任务设置的影响。最近的研究表明，经过了高质量的预训练与指令微调后，即使较小的语言模型（如 LLaMA-2 (7B)）也能够一定程度上展现出上述提到的三种能力，并且对于参数规模的要求随着预训练数据规模的扩展以及数据质量的提升在不断下降。此外，现有的研究对于能力涌现的实验往往局限于少数几个模型规模。例如，PaLM 模型的相关公开研究只在 8B、62B 和 540B 三种模型规模上进行了测试，对于未测试过的模型规模的性能尚不清楚。

2.3.2 涌现能力与扩展法则的关系

扩展法则和涌现能力提供了两种不同观点来理解大模型相对于小模型的优势，但是刻画了较为不同的扩展效应趋势。扩展法则使用语言建模损失来衡量语言模型的整体性能，整体上展现出了较为平滑的性能提升趋势，具有较好的可预测性，但是指数形式暗示着可能存在的边际效益递减现象；而涌现能力通常使用任务性能来衡量模型性能，整体上展现出随规模扩展的骤然跃升趋势，不具有可预测性，但是一旦出现涌现能力则意味着模型性能将会产生大幅跃升。由于这两种观点反映了不同的模型性能提升趋势（持续改进 v.s. 性能跃升），可能在一些情况下会导致不一致的发现与结论。

关于涌现能力的合理性也存在广泛的争议。一种推测是，涌现能力可能部分归因于特殊任务的设置 [43, 44]：现有评测设置中通常采用不连续的评估指标（如生成代码的准确性使用测试数据通过率评估）以及较为有限的模型参数规模（如 PaLM 技术报告里只展示了 8B、62B 和 540B 三个版本的模型）。在上述这两种情况下，很容易在下游任务的评测效果上产生不连续的变化趋势，导致了所谓的模型能力的“涌现现象”。特别地，如果针对性地修改评估指标时，或者提供更为连续的模型尺寸候选使之变得更为平滑后，涌现能力曲线的突然跃升趋势有可能会消失。这种分析一定程度上解释了模型性能的陡然跃升现象，为涌现能力的存在性打上了问号。然而，在实际使用中，用户就是以一种“不连续”的方式去感知大语言模型的性能优劣。换句话说，模型输出的正确性更为重要，用户满意度的体验过程本身就是离散的。例如，用户更倾向于使用能够正确通过所有测试用例的代码，而不愿意在两个失败代码之间选择一个包含错误较少的代码。

目前还缺少对于大语言模型涌现机理的基础性解释研究工作。与这一问题较

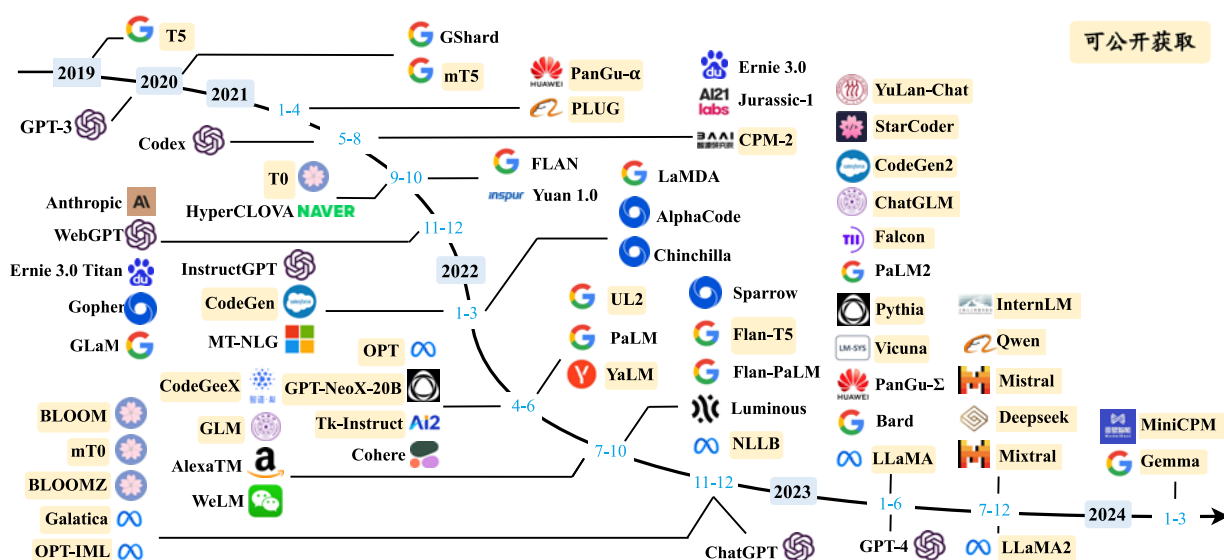


图 2.1 大语言模型发展时间线 (图片来源: [10])

为相关的研究叫做“顿悟”(Grokking),是指训练过程中的一种数据学习模式:模型性能从随机水平提升为高度泛化[45]。在未来研究中,还需要更为深入的相关讨论,才能够有效解释大模型的涌现机理。通俗来讲,扩展法则与涌现能力之间微妙的关系可以类比人类的学习能力来解释。以语言能力为例,对于儿童来说,语言发展(尤其是婴儿)可以被看作一个多阶段的发展过程,其中也会出现“涌现现象”。在这一发展过程中,语言能力在一个阶段内部相对稳定,但是当进入另一个能力阶段时可能会出现重要的提升(例如从说简单的单词到说简单的句子)。尽管儿童实际上每天都在成长,但是语言的提升过程本质上是不平滑和不稳定的(即语言能力在时间上不以恒定速率发展)。因此,经常可以看到年轻的父母会对宝宝所展现出的语言能力进展感到惊讶。

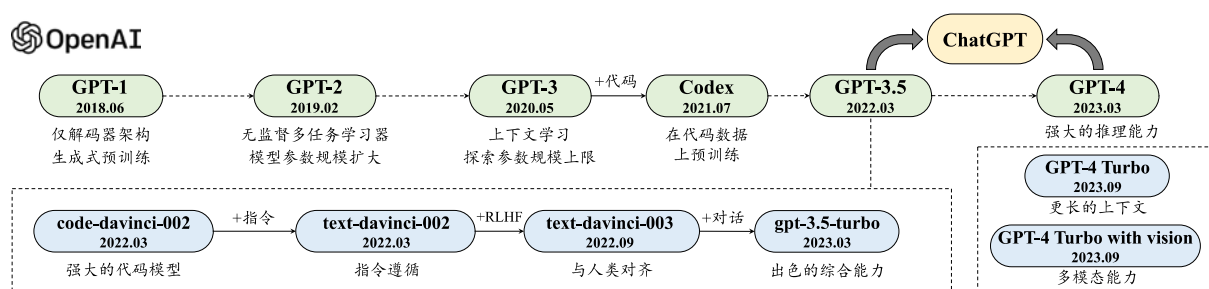


图 2.2 GPT 系列模型技术发展的历程图（图片来源：[10]）

2.4 GPT 系列模型的技术演变

2022 年 11 月底, OpenAI 推出了基于大语言模型的在线对话应用 — ChatGPT。由于具备出色的人机对话能力和任务解决能力, ChatGPT 一经发布就引发了全社会对于大语言模型的广泛关注, 众多的大语言模型应运而生, 并且数量还在不断增加 (图 2.1)。由于 GPT 系列模型具有重要的代表性, 本部分内容将针对 GPT 系列模型的发展历程进行介绍, 并且凝练出其中的重要技术创新。

GPT 系列模型的基本原理是训练模型学习恢复预训练文本数据, 将广泛的世界知识压缩到仅包含解码器 (Decoder-Only) 的 Transformer 模型中, 从而使模型能够学习获得较为全面的能力。其中, 两个关键要素是: (I) 训练能够准确预测下一个词的 Transformer (只包含解码器) 语言模型; (II) 扩展语言模型的规模以及扩展预训练数据的规模。图 2.2 展示了 GPT 系列模型的技术演进示意图, 这里主要根据 OpenAI 的论文、博客文章和官方 API 说明的信息进行绘制。该图中 实线表示在两个模型之间的进化路径上存在明确的证据 (例如, 官方声明新模型是基于基础模型开发的), 而 虚线 表示相对较弱的进化关系。截止到目前, OpenAI 对大语言模型的研发历程大致可分为四个阶段: 早期探索阶段、路线确立阶段、能力增强阶段以及能力跃升阶段, 下面进行具体介绍²。

2.4.1 早期探索

根据对于 Ilya Sutskever (OpenAI 联合创始人、前首席科学家) 的采访³, OpenAI 在成立初期就尝试使用语言模型研发人工智能系统, 但当时使用的是循环神经网络 [46], 模型能力和并行训练能力还存在较大的局限性。Transformer 刚刚问世, 就引起了 OpenAI 团队的高度关注, 并且将语言模型的研发工作切换到 Transformer 架构上, 相继推出了两个初始的 GPT 模型, 即 GPT-1 [14] 和 GPT-2 [17], 这两个早期工作奠定了后续更强大的 GPT 模型 (如 GPT-3 和 GPT-4) 的研究基础。

- *GPT-1*. 2017 年, Google 推出 Transformer 模型后, OpenAI 团队马上意识到这种神经网络架构将显著优于传统序列神经网络的性能, 有可能对于研发大型神经网络产生重要的影响。他们很快着手使用 Transformer 架构研发语言模型, 并于 2018 年发布了第一个 GPT 模型, 即 GPT-1, 模型名称 GPT 是生成式预训练

²本部分讨论的内容是作者基于调研 OpenAI 发布的论文、博客文章、采访报道和 API 说明文档所形成的个人理解。

³<https://hackernoon.com/an-interview-with-ilya-sutskever-co-founder-of-openai>

(Generative Pre-Training) 的缩写。GPT-1 基于生成式、仅有解码器的 Transformer 架构开发，奠定了 GPT 系列模型的核心架构与基于自然语言文本的预训练方式，即预测下一个词元。由于当时模型的参数规模还相对较小，模型仍然缺乏通用的任务求解能力，因而采用了无监督预训练和有监督微调相结合的范式。与 GPT-1 同期发布的预训练语言模型是大名鼎鼎的 BERT 模型。BERT 与 GPT-1 虽然都采用了基于 Transformer 架构的预训练学习方式，但是它主要面向自然语言理解任务 (Natural Language Understanding, NLU)，为此只保留了 Transformer 中的编码器，其中 BERT-Large 模型在众多的自然语言理解任务上取得了非常重要的提升，成为当时备受瞩目的“明星模型”。可以说，BERT 当时引领了自然语言处理社区的研究浪潮，涌现了大量针对它改进与探索的工作。由于 GPT-1 模型规模实际上与小规模的 BERT-Base 模型相当 (100M 左右参数)，在公开评测数据集合上的性能尚不能达到当时众多竞争模型中的最优效果，没有引起学术界的足够关注。

• **GPT-2.** GPT-2 沿用了 GPT-1 的类似架构，将参数规模扩大到 1.5B，并使用大规模网页数据集 WebText 进行预训练。与 GPT-1 不同，GPT-2 旨在探索通过扩大模型参数规模来提升模型性能，并且尝试去除针对特定任务所需要的微调环节。这点在 GPT-2 的论文 [17] 中得到了着重论述，它试图使用无监督预训练的语言模型来解决各种下游任务，进而不需要使用标注数据进行显式的模型微调。形式化来说，多任务学习 (Multi-task Learning) 可以通过一种较为通用的概率形式刻画，即 $P(\text{output}|\text{input}, \text{task})$ ——根据输入和任务信息来预测输出。为了建立通用的多任务学习框架，GPT 系列模型将输入、输出和任务信息都通过自然语言形式进行描述，进而后续任务的求解过程就可以看作是任务方案（或答案）的文本生成问题。OpenAI 团队在 GPT-2 的论文中还尝试解释无监督预训练在下游任务中取得良好效果的原因：“由于特定任务的有监督学习目标与无监督学习目标（语言建模）在本质上是相同的（预测下一个词元），主要区别就在于它们只是在全部训练数据的子集上进行优化，因此对于特定下游任务而言，优化无监督的全局学习目标本质上也是在优化有监督的任务学习目标” [17]。对这一说法的通俗理解是，语言模型将每个（自然语言处理）任务都视为基于世界文本子集的下一个词预测问题。因此，如果无监督语言建模经过训练后具有足够的能力复原全部世界文本，那么本质上它就能够解决各种任务。这些 GPT-2 论文中的早期讨论与 Ilya Sutskever 在接受 Jensen Huang 采访时的观点非常类似：“神经网络学到的是生成文本的过程中的某种表示，这些模型的生成文本实际上是真实世界的投影……（语言模型）对

下一个单词的预测越准确，（对于世界知识）保真度就越高，在这个过程中获得的分辨率就越高……”⁴。

2.4.2 规模扩展

虽然 GPT-2 的初衷是成为一个“无监督多任务学习器”，但在很多任务上与有监督微调方法相比，模型效果整体上还是要逊色一些。在 GPT-2 基础上，GPT-3 针对（几乎相同的）模型参数规模进行了大幅扩展，在下游任务中初步展现出了一定的通用性（通过上下文学习技术适配下游任务），为后续打造更为强大的模型确立了关键的技术发展路线。

• **GPT-3.** OpenAI 在 2020 年发布了 GPT-3 模型，将模型参数扩展到了 175B 的规模。与 GPT-2 相比，GPT-3 直接将参数规模提升了 100 余倍，对于模型扩展在当时给出了一个极限尝试，其雄心、魄力可见一斑。值得一提的是，OpenAI 的两篇关于扩展法则的论文 [15, 21] 都是在 2020 年发表的，这说明在 GPT-3 开始训练时可能已经进行了比较充分的实验探索，包括小版本模型的尝试、数据收集与清洗、并行训练技巧等。在 GPT-3 的论文中，它正式提出了“上下文学习”这一概念，使得大语言模型可以通过少样本学习的方式来解决各种任务。上下文学习可以指导大语言模型学会“理解”自然语言文本形式描述的新任务，从而消除了针对新任务进行微调的需要。基于这一学习范式，大语言模型的训练与利用可以通过语言建模的形式进行统一描述：模型预训练是在给定上下文条件下预测后续文本序列，模型使用则是根据任务描述以及示例数据来推理正确的任务解决方案。GPT-3 不仅在各种自然语言处理任务中表现出了优异的效果，对于一些需要复杂推理能力或领域适配能力的特定任务也具有较好的解决能力。虽然 GPT-3 的论文没有明确提出上下文学习能力是大语言模型的涌现能力，但是指出了上下文学习对于大模型的性能增益会更加显著，而对于小模型来说则收益较小（见 GPT-3 论文 [23] 的原始图 1.2）。总体而言，GPT-3 可以被看作从预训练语言模型到大语言模型演进过程中的一个重要里程碑，它证明了将神经网络扩展到超大规模可以带来大幅的模型性能提升，并且建立了以提示学习方法为基础技术路线的任务求解范式。

⁴<https://lifearchitect.ai/ilya/>

2.4.3 能力增强

由于具有较强的模型性能，GPT-3 成为 OpenAI 开发更强大的大语言模型的研究基础。根据公开资料披露的内容来说，OpenAI 探索了两种主要途径来改进 GPT-3 模型，即代码数据训练和人类偏好对齐。

- 代码数据训练. 原始的 GPT-3 模型的复杂推理任务能力仍然较弱，如对于编程问题和数学问题的求解效果不好。为了解决这一问题，OpenAI 于 2021 年 7 月推出了 Codex [47]，这是一个在大量 GitHub 代码数据集上微调的 GPT 模型。实验结果表明，Codex 可以解决非常困难的编程问题，还能显著提升大模型解决数学问题的能力 [48]。此外，2022 年 1 月 OpenAI 还公开了一种用于训练文本和代码嵌入的对比方法 [49]，结果表明该方法能够改善一系列相关任务的性能，包括线性探测分类、文本搜索和代码搜索等。根据 OpenAI 所发布的 API 信息所示，GPT-3.5 模型是在基于代码训练的 GPT 模型（即 `code-davinci-002`）基础上开发的，这表明在代码数据上进行训练有助于提高 GPT 模型的综合性能，尤其是代码能力。另一个可能的启发是对于可用于预训练的数据范围的扩展，可能并不局限于自然语言形式表达的文本数据。

- 人类对齐. OpenAI 关于人类对齐的公开研究工作可以追溯到 2017 年（实际时间或更早）。在一篇题为“Learning from Human Preferences”⁵的博客文章中，OpenAI 的研究团队介绍了一项使用强化学习算法从人类标注的偏好数据中学习如何改进模型性能的工作 [50]。在这篇强化学习工作发表不久，2017 年 7 月 OpenAI 研究团队又提出了一项改进的强化学习算法—PPO 算法（Proximal Policy Optimization, PPO）[51]，这也成为了 OpenAI 在后续人类对齐技术里所采用的标配强化学习算法。在 2020 年，OpenAI 研究团队将人类对齐算法应用于提升自然语言处理任务上的能力，训练了一个根据人类偏好进行优化的摘要模型 [52]。以这些前期工作为基础，2022 年 1 月，OpenAI 正式推出 InstructGPT [28] 这一具有重要影响力的学术工作，旨在改进 GPT-3 模型与人类对齐的能力，正式建立了基于人类反馈的强化学习算法，即 RLHF 算法。值得一提的是，在 OpenAI 的论文和相关文档中，很少使用“指令微调”（Instruction Tuning）一词，主要是使用“监督微调”一词（即基于人类反馈的强化学习算法的第一步 [28]）。除了提高指令遵循能力，基于人类反馈的强化学习算法有助于缓解有害内容的生成，这对于大语言模型在实际应用中的安全部署非常重要。OpenAI 在一篇技术博客文章中描述了他们对齐

⁵<https://openai.com/research/learning-from-human-preferences>

研究的技术路线 [53]，并总结了三个有前景的研究方向：训练人工智能系统以达到（1）使用人类反馈、（2）协助人类评估和（3）进行对齐研究。

通过这些增强技术，OpenAI 将改进后的具有更强能力的 GPT 模型命名为 GPT-3.5 模型（参见第 3.1 节中有关 OpenAI API 的讨论）。

2.4.4 性能跃升

在历经上述近五年的重要探索，OpenAI 自 2022 年底开始发布了一系列重要的技术升级，其中具有代表性的模型是 ChatGPT、GPT-4 以及 GPT-4V/GPT-4 Turbo，这些模型极大提高了现有人工智能系统的能力水平，成为了大模型发展历程中的重要里程碑。

- *ChatGPT*. 2022 年 11 月，OpenAI 发布了基于 GPT 模型的人工智能对话应用服务 ChatGPT。OpenAI 官方博客文章 [54] 概要地介绍了 ChatGPT 的研发技术，主要是沿用了 InstructGPT（原帖中称 ChatGPT 为 “InstructGPT 的兄弟模型”）的训练技术，但是对于对话能力进行了针对性优化。在训练数据的收集过程中，ChatGPT 将人类生成的对话数据（同时扮演用户和人工智能的角色）与训练 InstructGPT 的相关数据进行结合，并统一成对话形式用于训练 ChatGPT。ChatGPT 在与人机对话测试中展现出了众多的优秀能力：拥有丰富的世界知识、复杂问题的求解能力、多轮对话的上下文追踪与建模能力、与人类价值观对齐的能力等。在后续的版本更迭中，ChatGPT 进一步又支持了插件机制，通过现有工具或应用程序扩展了它的功能，能够超越以往所有人机对话系统的能力水平。ChatGPT 一经推出就引发了社会的高度关注，对于人工智能的未来研究产生了重要影响。

- *GPT-4*. 继 ChatGPT 后，OpenAI 于 2023 年 3 月发布了 GPT-4 [35]，它首次将 GPT 系列模型的输入由单一文本模态扩展到了图文双模态。总体来说，GPT-4 在解决复杂任务方面的能力显著强于 GPT-3.5，在一系列面向人类的考试中都获得了非常优异的结果。GPT-4 发布后，微软的研究团队针对其进行了大规模人类生成问题的性能测试 [20]，实验结果表明 GPT-4 具有令人震撼的模型性能，论文作者认为 GPT-4 的到来展现出了通用人工智能的曙光。此外，由于进行了为期六个月的迭代对齐（在基于人类反馈的强化学习中额外增加了安全奖励信号），GPT-4 对恶意或挑衅性查询的响应更加安全。在技术报告中 [35]，OpenAI 强调了安全开发 GPT-4 的重要性，并应用了一些干预策略来缓解大语言模型可能出现的问题——幻觉、隐私泄露等。例如，研究人员引入了“红队攻击”（Red Teaming）机制 [55]

来减少生成有害或有毒的内容。更重要的是，GPT-4 搭建了完备的深度学习训练基础架构，进一步引入了可预测扩展的训练机制，可以在模型训练过程中通过较少计算开销来准确预测模型的最终性能。

● *GPT-4V*、*GPT-4 Turbo* 以及多模态支持模型. 基于发布的 GPT-4 初版模型 [35], OpenAI 在 2023 年 9 月进一步发布了 GPT-4V, 重点关注 GPT-4 视觉能力的安全部署。在 GPT-4V 的系统说明中 [56], 广泛讨论了与视觉输入相关的风险评估手段和缓解策略。GPT-4V 在多种应用场景中表现出了强大的视觉能力与综合任务解决能力。在 2023 年 11 月, OpenAI 在开发者大会上发布了升级版的 GPT-4 模型, 称为 *GPT-4 Turbo*, 引入了一系列技术升级: 提升了模型的整体能力 (比 GPT-4 更强大), 扩展了知识来源 (拓展到 2023 年 4 月), 支持更长上下文窗口 (达到 128K), 优化了模型性能 (价格更便宜), 引入了若干新的功能 (如函数调用、可重复输出等)。同时, Assistants API 功能也被推出, 旨在提升人工智能应用助手的开发效率, 开发人员可以利用特定的指令、外部知识和工具, 在应用程序中快速创建面向特定任务目标的智能助手。此外, 新版本的 GPT 模型还进一步增强了多模态能力, 分别由 GPT-4 Turbo with Vision、DALL·E-3、TTS (Text-to-speech) 以及 Listen to voice samples 等支持实现。这些技术升级进一步提高了 GPT 模型的任务性能, 扩展了 GPT 模型的能力范围。更重要的是, 随着模型性能和支撑功能的改进, 极大地加强了以 GPT 模型所形成的大模型应用生态系统。

尽管 GPT 系列模型取得了巨大的科研进展, 这些最前沿的大语言模型仍然存在一定的局限性。例如, GPT 模型可能在某些特定上下文中生成带有事实错误的内容 (即幻觉) 或存在潜在风险的回应 [35]。更多针对大语言模型局限性的讨论将在本书的第 12 章进行详细讨论。从人工智能的发展历程来看, 开发能力更强、更安全的大语言模型是一项长期的研究挑战。为了有效降低使用模型的潜在风险, OpenAI 采用了迭代部署策略 [57], 通过多阶段开发和部署的生命周期来研发模型与产品。